Memoria técnica de BBK BetCamp

1. Arquitectura y flujo de datos

El código está organizado para que se pueda leer siguiendo el funcionamiento del juego. Primero se declaran las referencias a la interfaz y las clases. Después se crean los participantes y la partida, y se definen las funciones y los eventos que conectan la lógica con la pantalla.

La clase **Jugador** guarda el nombre, las cartas y los puntos de cada participante. Tanto el jugador como la banca usan esta clase porque ambos reciben cartas y calculan su puntuación con las mismas reglas. La clase **Partida** reúne a los participantes y conserva la baraja, el estado de la mano y su resultado.

Al pulsar **Repartir**, el juego solicita una baraja a Deck of Cards API y reparte dos cartas a cada participante. Durante la mano, JavaScript actualiza las cartas y los marcadores en pantalla. Al terminar, guarda el resultado en el almacenamiento local del navegador y actualiza el historial, que se puede filtrar por nombre.

2. Diseño e interfaz

Busqué una interfaz llamativa que invitara a jugar y ayudara a seguir la mano. El tapete verde delimita la zona de juego, las cartas grandes permiten distinguir las dos manos de un vistazo y los botones indican qué acciones están disponibles en cada turno.

El crupier acompaña la partida con un bocadillo que cambia según lo que ocurre en la mesa. Sus mensajes indican cuándo se reparten las cartas, cuándo le toca decidir al jugador y cuándo juega la banca. Al terminar la mano, el fondo se oscurece para centrar la atención en el resultado y en las dos opciones disponibles, **Otra mano** y **Dejar la mesa**.

3. Gestión del trabajo

El proyecto fue individual, así que me encargué del diseño, la programación y la documentación. Organicé el trabajo en cuatro fases.

1. **Exploración.** Probé otra API y planteé un proyecto más amplio que acabé descartando.
2. **Cambio de proyecto.** Redefiní la entrega en torno a un juego de cartas y limité el alcance a una mesa de blackjack.
3. **Desarrollo.** Implementé el reparto, los turnos, el cálculo de puntos y el historial.
4. **Cierre.** Ajusté el diseño de la mesa, los mensajes del crupier y el aviso de fin de mano.

La gestión del tiempo ha sido el punto más flojo. Dediqué demasiado esfuerzo a una API que finalmente descarté y, con el plazo ya ajustado, he seguido planteando ampliaciones que podían esperar. Me habría convenido definir antes qué funciones necesitaba para cerrar una primera versión jugable.

4. Conclusiones

El mayor desafío técnico ha sido reorganizar el código mientras todavía estaba desarrollando el juego. Quería evitar duplicar lógica y facilitar futuras ampliaciones. Separar **Jugador** de **Partida** me permitió usar el mismo cálculo de puntos para el jugador y la banca. También agrupé a los participantes en una lista para que el reparto recorra sus manos con la misma lógica.

La versión actual permite que una persona juegue contra la banca. Si más adelante añado jugadores, podré reutilizar el modelo de participantes y el reparto, aunque tendré que adaptar los turnos, la interfaz y la resolución de cada mano.

Para el próximo proyecto, mi prioridad será cerrar primero una versión jugable de principio a fin. A partir de ahí, podré decidir qué cambios de estructura merece la pena hacer según la siguiente mejora concreta.